

GUIA DIDACTICA PARA EXPLICACION TECNICA

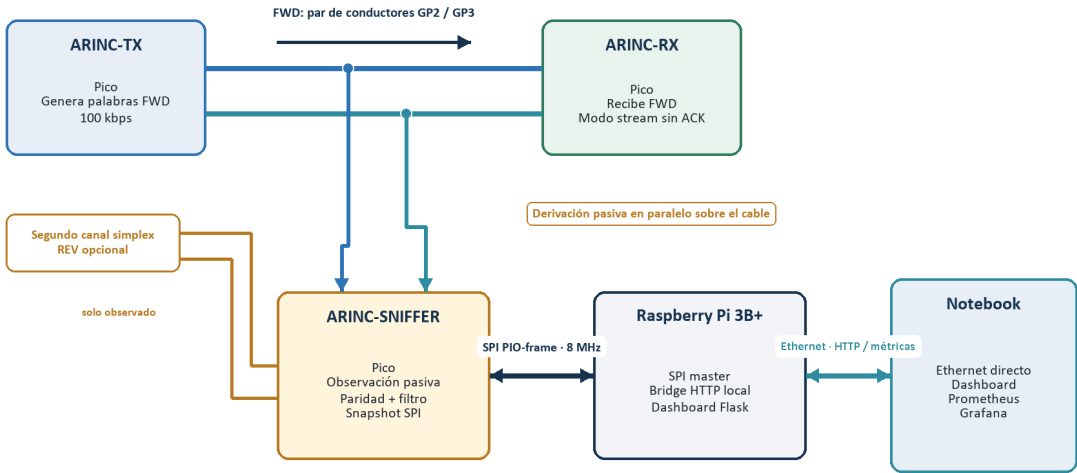
PAMPA / ARINC 429 - de la palabra al dashboard

Resumen para explicar el sistema completo con lenguaje simple, sin perder precision tecnica.

Fase	Rama	Modo recomendado	Estado
arinc429_logic	ARINC	stream + sniffer PIO-frame 8 MHz	validado

Arquitectura completa

Flujo de datos desde la generación lógica hasta la observabilidad



Cadena completa: TX genera, RX recibe, SNIFFER observa, 3B+ publica, notebook visualiza.

Idea central

El proyecto no intenta decir que un GPIO sea una línea ARINC real. Lo que se validó es la lógica de palabra, la temporización de 100 kbps, la captura pasiva, la integridad por paridad, el transporte SPI por trama completa y la visualización.

1. Lectura de extremo a extremo

Esta es la explicación corta del recorrido de una palabra desde que nace hasta que aparece en pantalla.

Flujo general

En criollo: TX arma una palabra, la saca por dos GPIO, RX la interpreta y el SNIFFER la escucha como si estuviera mirando el cable desde el costado.

Detalle técnico: FWD es el canal simplex principal. El SNIFFER toma FWD/REV en paralelo y no transmite sobre esas líneas; solamente responde por SPI a la 3B+.

Frase defendible: El SNIFFER no conversa con TX ni con RX por ARINC: solo observa el cable y reporta por otro enlace.

Por que hay Raspberry Pi 3B+

En criollo: La Pico SNIFFER se concentra en capturar. La 3B+ hace de puente: pregunta por SPI, convierte a JSON y deja los datos disponibles.

Detalle técnico: El bridge HTTP escucha en 127.0.0.1:5100 dentro de la 3B+. Flask consulta ese bridge local y expone el dashboard en 192.168.50.2:5000.

Frase defendible: Separamos adquisición dura de visualización: la Pico captura, la 3B+ sirve datos, la notebook mira.

Por que Ethernet directo

En criollo: Con Wi-Fi aparecía más variación y más riesgo de saturar la 3B+. Con cable directo queda más estable y repetible.

Detalle técnico: La red recomendada es notebook 192.168.50.1/24 y 3B+ 192.168.50.2/24. Solo Flask queda visible hacia la notebook.

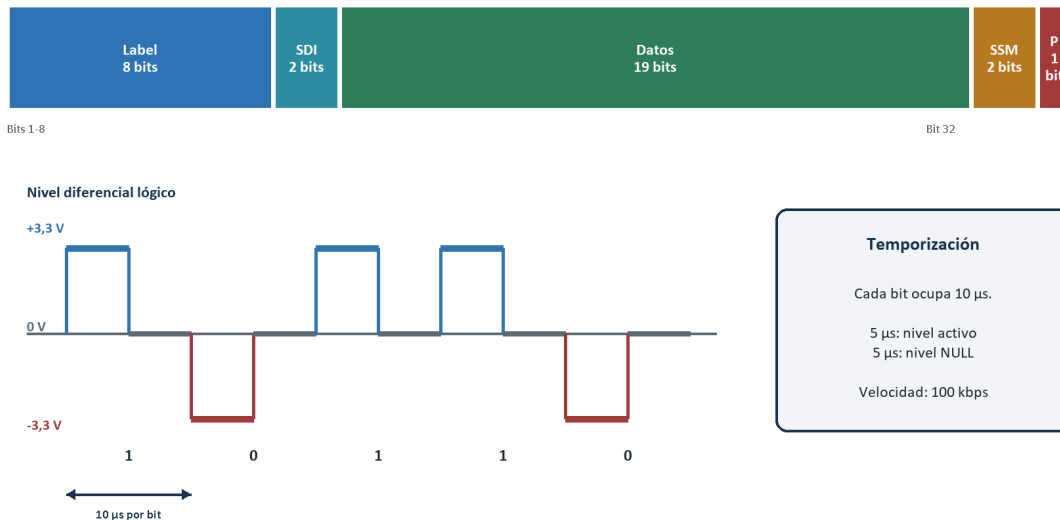
Frase defendible: El enlace operativo no depende de Internet ni del router: es una red local corta y controlada.

2. La palabra ARINC logica

Antes de hablar de cables, el sistema arma una palabra de 32 bits con campos definidos.

Modelo lógico ARINC 429

Palabra de 32 bits y codificación bipolar con retorno a cero.



Palabra de 32 bits y bit time de 10 us con retorno a cero.

Como nace la palabra

En criollo: Una palabra es un paquete chico de 32 bits: trae un nombre de variable, un identificador, el dato, estado y paridad.

Detalle tecnico: Los campos usados son Label, SDI, DATA, SSM y paridad impar. Ejemplos utiles: 0xA5 temperatura, 0xB1 velocidad, 0xC2 altitud.

Frase defendible: No se manda texto por el cable: se manda una palabra binaria que despues se interpreta por label y SDI.

Para que sirve la paridad

En criollo: La paridad es un control rapido: si una palabra llega con un bit mal, queda marcada como invalida.

Detalle tecnico: En la version endurecida, una palabra con mala paridad suma error, pero no entra a `accepted_words` ni actualiza el snapshot.

Frase defendible: El sistema prefiere perder una palabra dudosa antes que mostrar un dato corrupto como valido.

3. ARINC-TX: generacion y transmision

El TX no dibuja pulsos desde la CPU bit por bit. La CPU arma palabras y el PIO mantiene el tiempo fino.

Flujo interno: ARINC-TX y ARINC-RX

Generación, serialización, FIFO y procesamiento de palabras

ARINC-TX



ARINC-RX



El modo stream evita lotes fijos: TX emite ráfagas de longitud variable y RX drena el FIFO continuamente. Las operaciones extensas se mantienen fuera del lazo de captura.

Camino interno resumido de TX y RX: CPU, FIFO, PIO y procesamiento.

Que hace TX

En criollo: TX inventa valores de laboratorio, por ejemplo temperatura, velocidad y altitud, y los empaqueta en palabras ARINC lógicas.

Detalle técnico: El target recomendado es `arinc_tx_arinc429_logic_stream`. Emite ráfagas variables y pausas variables; no depende de bloques fijos de 1000 palabras.

Frase defendible: La transmisión actual es stream: no espera ACK para seguir funcionando.

Por que usamos PIO

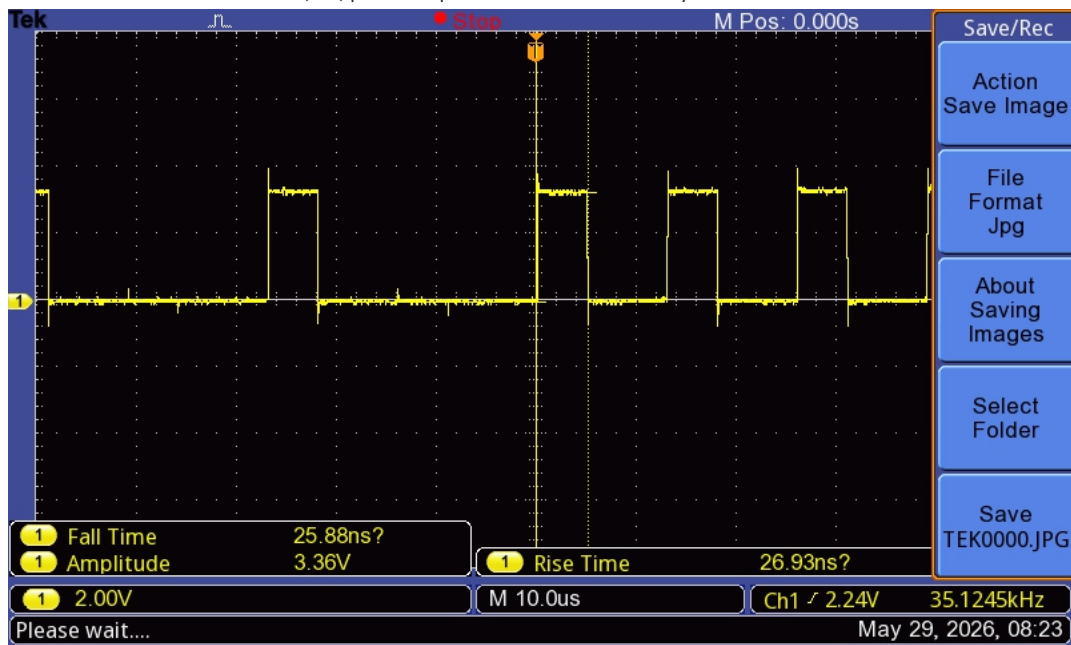
En criollo: PIO es como un asistente de hardware dentro de la Pico: una vez cargada la palabra, mantiene los tiempos sin distraerse.

Detalle técnico: La CPU coloca palabras en el TX FIFO; la state machine PIO usa OSR/X/Y para desplazar bit por bit a 100 kbps.

Frase defendible: La temporización de 10 us por bit no queda atada a la velocidad variable del código C.

4. Línea FWD y codificación bipolar RZ

En esta fase el nivel eléctrico es GPIO de 3,3 V, pero se reproduce la idea diferencial y el retorno a cero.



Canal individual respecto de masa: pulso de 0 a 3,3 V y retorno a cero.

Que significa retorno a cero

En criollo: Cada bit no queda alto durante todo el tiempo. Primero aparece el pulso activo y despues vuelve a cero antes del siguiente bit.

Detalle tecnico: Cada bit dura 10 us. Durante 5 us hay nivel activo y durante 5 us hay NULL. Por eso la captura muestra pulsos de ancho cercano a 5 us.

Frase defendible: El cero entre pulsos no es falta de señal: es parte de la codificación RZ.

Por que hay dos conductores

En criollo: La informacion sale como diferencia entre dos lineas. A veces domina una, a veces domina la otra, y a veces ambas quedan en cero.

Detalle tecnico: En laboratorio se implementa con dos GPIO complementarios. En MATH = CH1 - CH2 aparecen niveles positivos, negativos y cero.

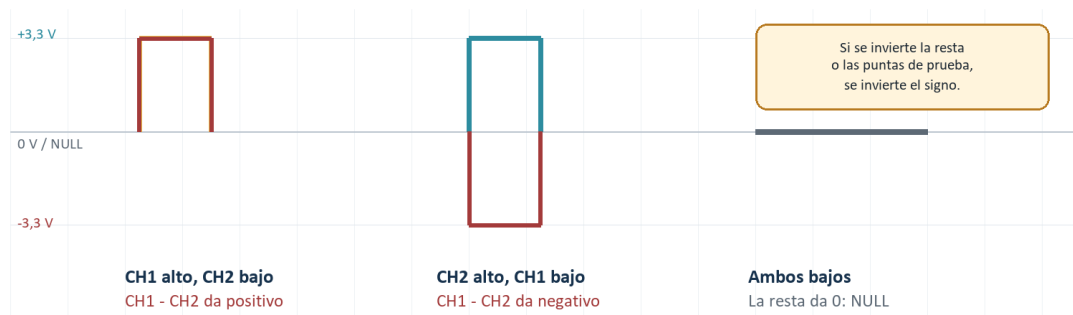
Frase defendible: Lo importante no es solo cada canal contra masa, sino la resta entre ambos.

5. Osciloscopio: rojo, azul y la resta CH1 - CH2

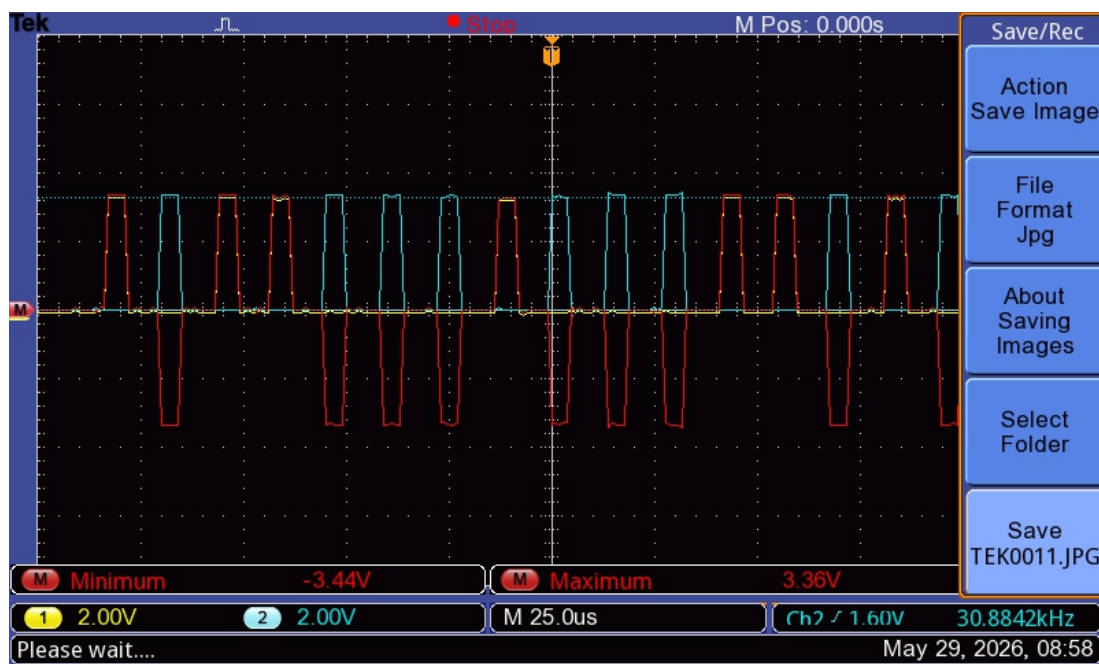
Esta es la explicación práctica para la duda de los pulsos espejados.

Como leer CH1, CH2 y MATH en el osciloscopio

El azul/celeste es un canal físico contra masa; el rojo es la resta matemática CH1 - CH2.



Guía de lectura de CH1, CH2 y MATH en el osciloscopio.



Captura real: MATH = CH1 - CH2 con niveles positivo, nulo y negativo.

Pulsos celestes y rojos

En criollo: El celeste es CH2 medido contra masa. El rojo es la cuenta que hace el osciloscopio: CH1 menos CH2.

Detalle técnico: Si CH2 está alto y CH1 bajo, CH1 - CH2 da negativo; por eso se ve celeste hacia arriba y rojo hacia abajo. Si CH1 está alto y CH2 bajo, la resta da positiva y el rojo va hacia arriba.

Frase defendible: El pulso celeste no es otra palabra: es una línea física. El rojo es la resta matemática. Positivo y negativo son las dos polaridades usadas para representar bits.

6. ARINC-RX: recepción y procesamiento

RX reconstruye la palabra desde la línea, valida y actualiza su estado interno.

Que hace RX

En criollo: RX mira el par FWD, detecta los pulsos, reconstruye la palabra y la interpreta.

Detalle técnico: El PIO RX detecta símbolos HI/LO/NULL, reconstruye 32 bits y empuja palabras al RX FIFO para que la CPU las procese.

Frase defendible: La CPU no tiene que adivinar el tiempo exacto de cada bit; recibe palabras reconstruidas desde el PIO.

Que es el FIFO

En criollo: El FIFO es una fila corta entre PIO y CPU. Sirve para que una palabra no se pierda si la CPU tarda muy poco en atender.

Detalle técnico: En RX conserva hasta 4 palabras. Por eso el lazo de captura debe drenar seguido y no hacer tareas largas como imprimir demasiado o procesar archivos.

Frase defendible: El FIFO no es una base de datos: es un amortiguador corto entre hardware y software.

REV y ACK

En criollo: En el modo histórico RX podía responder ACK por REV. En el modo stream final no hace falta: RX recibe y procesa sin devolver confirmación.

Detalle técnico: ARINC 429 real es simplex. Si un equipo necesita responder, normalmente usa otro canal físico en sentido inverso.

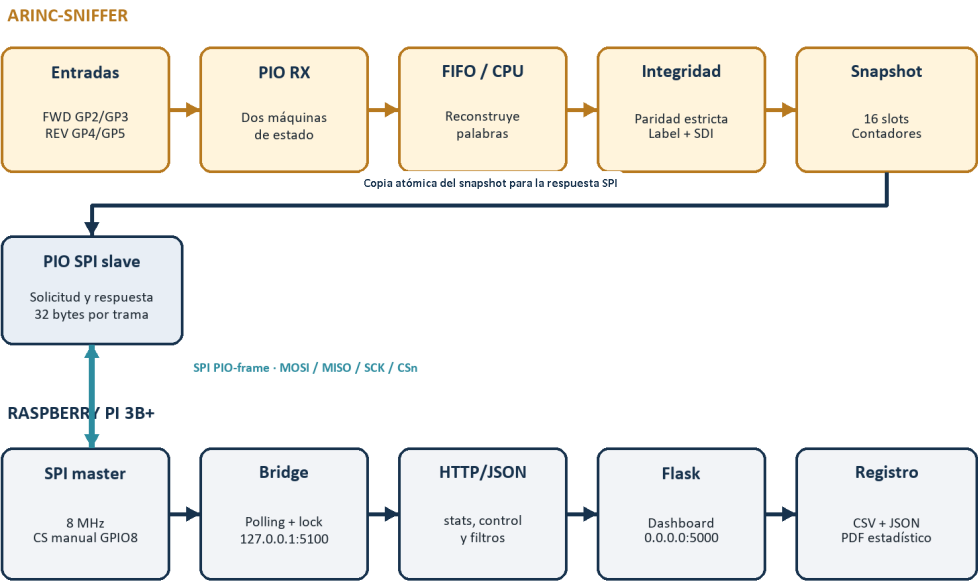
Frase defendible: REV no es el mismo bus hablando al revés: es otro enlace simplex, y en el modo final queda inactivo.

7. ARINC-SNIFFER: captura pasiva

El SNIFFER es la pieza central del proyecto: observa sin intervenir y entrega una vista limpia de los datos.

Flujo interno: SNIFFER y Raspberry Pi 3B+

Captura pasiva, clasificación, snapshot, SPI y servicios de visualización



Ruta interna: captura PIO, paridad, filtro, snapshot, SPI y bridge.

Donde se conecta el SNIFFER

En criollo: Se conecta al cable en paralelo. No va 'entre' TX y RX como si cortara la línea.

Detalle técnico: Sus entradas GP2/GP3 observan FWD y GP4/GP5 observan REV opcional. No transmite en esos pines.

Frase defendible: Es un observador de alta prioridad lógica: escucha todo lo que puede, pero no modifica el enlace.

Filtro Label/SDI

En criollo: No todo lo que pasa por el cable interesa. El filtro deja pasar solo combinaciones conocidas.

Detalle técnico: La whitelist actual acepta 0xA5/0, 0xB1/1, 0xC2/2 y conserva 0xAC/3 como ACK histórico.

Frase defendible: Una palabra puede estar bien formada y aun así ser filtrada si no pertenece al conjunto de interés.

Snapshot

En criollo: En vez de mandar millones de palabras a la 3B+, el SNIFFER conserva el último valor y contadores por variable.

Detalle técnico: El snapshot tiene 16 slots. Cada slot se identifica por canal, label y SDI, e incluye valor, SSM, contador y revisión.

Frase defendible: La 3B+ consulta estado reciente y contadores; no necesita copiar cada palabra cruda.

8. SPI entre SNIFFER y Raspberry Pi 3B+

SPI no reemplaza a ARINC. Es solo el enlace interno entre el sniffer y el host.

Por que SPI a 8 MHz si ARINC va a 100 kbps

En criollo: Son enlaces distintos. ARINC es el cable observado; SPI es el camino para sacar datos del SNIFFER hacia la 3B+.

Detalle tecnico: SPI a 8 MHz da margen para polling, respuestas de 32 bytes, estadísticas y dashboard sin apretar el sistema.

Frase defendible: Que ARINC sea 100 kbps no obliga a que SPI también sea 100 kbps.

Por que no 50 MHz

En criollo: Porque el máximo teórico no es lo mismo que el máximo estable del sistema real con cables, PIO, CS manual y Linux.

Detalle tecnico: El barrido mostro 8 y 9 MHz PASS, 10 MHz parcial, 16 y 50 MHz FAIL. Por margen se eligió 8 MHz.

Frase defendible: La frecuencia correcta es la más rápida estable, no la más grande que aparece en una hoja de datos.

Trama completa, no byte suelto

En criollo: La objeción del byte a byte se resolvió usando PIO-frame: el intercambio se organiza como paquetes de 32 bytes.

Detalle tecnico: El host manda request de 32 bytes, espera 2 ms y luego relojea response de 32 bytes. El slave SPI está implementado con PIO.

Frase defendible: La unidad lógica de SPI ahora es la trama completa de protocolo.

Errores SPI de arranque

En criollo: Si el bridge arranca antes que la Pico, puede consultar cuando todavía no hay respuesta válida. Eso no es un error operativo del enlace.

Detalle tecnico: Se separaron spi_startup_errors y spi_errors. spi_errors empieza a contar después de la primera respuesta SPI válida.

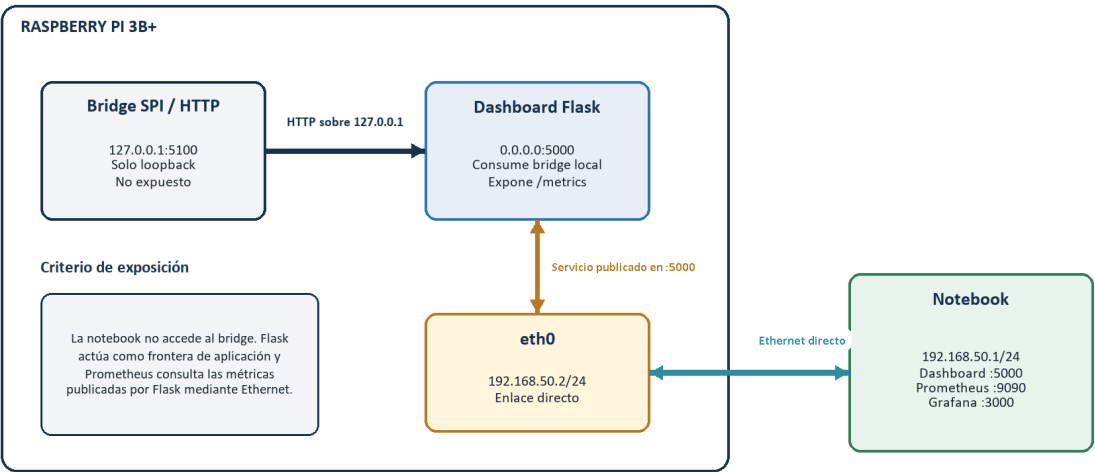
Frase defendible: La corrida final tuvo spi_errors operativos en cero.

9. Bridge, Flask, Prometheus y Grafana

La visualización esta separada en una vista operativa rapida y una vista historica.

Red y servicios

Separación entre el transporte local y la interfaz expuesta a la notebook



Servicios y direcciones IP recomendadas.

Bridge HTTP
En criollo: El bridge es un traductor local: habla SPI con el SNIFFER y HTTP/JSON con Flask.
Detalle tecnico: Escucha en 127.0.0.1:5100. Eso significa que queda dentro de la 3B+ y no se publica directamente a la notebook.
Frase defendible: El control bajo nivel de SPI queda encerrado en la Raspberry.

Dashboard Flask
En criollo: Flask es la pantalla de uso diario. Muestra valores actuales, contadores, errores y estado del enlace.
Detalle tecnico: Corre en 0.0.0.0:5000 para que la notebook entre por http://192.168.50.2:5000.
Frase defendible: Flask es la vista operativa rapida: sirve para mirar si el sistema esta vivo y sano.

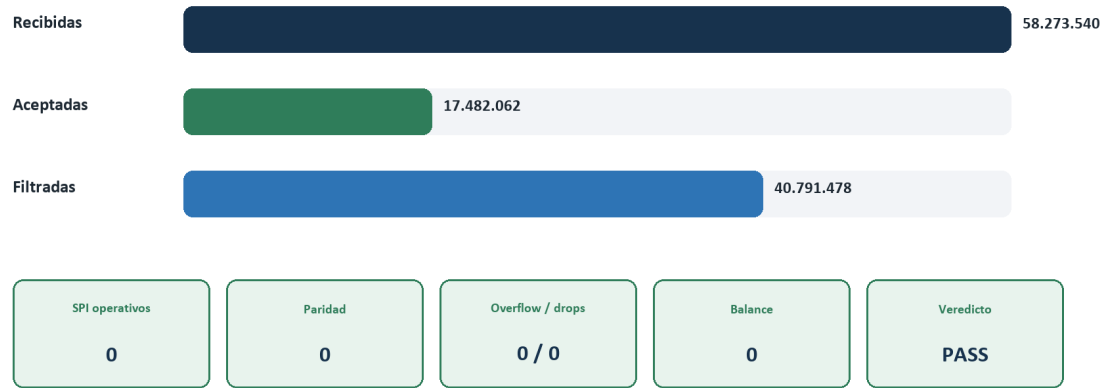
Prometheus y Grafana
En criollo: Prometheus guarda series temporales y Grafana las grafica. No reemplazan al dashboard, lo complementan.
Detalle tecnico: Prometheus consulta /metrics de Flask desde la notebook y Grafana usa esos datos para historico.
Frase defendible: Flask responde que pasa ahora; Prometheus/Grafana responden que paso durante horas.

10. Estadística, exportación y evidencias

El proyecto no depende solo de mirar la pantalla: genera evidencia exportable.

Validación final de ocho horas

Sesión 20260606_022347_corrida_final_stream_8mhz_8h



Resumen visual de contadores de la corrida final.

Prueba	Resultado principal	Lectura
Corrida final 8 h	58.273.540 recibidas, 17.482.062 aceptadas, 40.791.478 filtradas	PASS
Errores operativos	spi_errors=0, parity_errors=0, overflow=0, drops=0	sin fallas de transporte
Paridad estricta	6.109 errores inyectados y descartados	filtro de integridad validado
Barrido SPI	8/9 MHz PASS, 10 MHz parcial, 16/50 MHz FAIL	8 MHz recomendado

stats_recorder

En criollo: Durante una prueba se puede dejar la 3B+ registrando muestras y eventos. Al cerrar, quedan archivos para revisar.

Detalle técnico: El recorder genera samples.csv, events.csv, summary.json y report.pdf. Eso permite analizar sin depender del chat ni de capturas manuales.

Frase defendible: La prueba deja evidencia reproducible: datos crudos, resumen y reporte.

11. Límites actuales y siguiente fase

Esta sección conviene decirlo con claridad: lo validado es fuerte, pero tiene frontera definida.

Que se validó

En criollo: Se validó la cadena lógica completa: palabra, temporización, captura pasiva, integridad, SPI, bridge, dashboard y observabilidad.

Detalle técnico: La fase final quedó reproducible con targets stream, sniffer PIO-frame, SPI a 8 MHz y red Ethernet directa.

Frase defendible: La capa digital del prototipo ya tiene una base estable para seguir.

Que no es todavía

En criollo: Todavía no es un sniffer plug-and-play para enchufar a un bus ARINC real de aviónica.

Detalle técnico: Faltan receptor ARINC 429 real, protección, entrada de alta impedancia, tolerancia a niveles de campo, conectores, masa/blindaje y validación eléctrica.

Frase defendible: No se debe conectar GP2/GP3 directo a una línea ARINC real.

Como sigue

En criollo: El siguiente paso es agregar un frente eléctrico real delante de la Pico SNIFFER, sin tirar lo ya hecho.

Detalle técnico: La captura, filtros, protocolo SPI, bridge, Flask, Prometheus, Grafana y recorder pueden reutilizarse detrás del receptor ARINC dedicado.

Frase defendible: La evolución natural es cambiar la entrada eléctrica, no reescribir todo el sistema.

Cierre defendible

La etapa actual demuestra que la arquitectura digital funciona y que el cuello pendiente está en la interfaz física ARINC real. Ese es el próximo desarrollo de hardware.

12. Respuestas cortas ante preguntas previsibles

Estas frases sirven como guía oral si el tutor apunta a puntos concretos.

Pregunta	Respuesta breve
Por que FWD y REV?	Porque son dos sentidos logicos. En ARINC real cada canal es simplex; si hay retorno se usa otro canal.
El SNIFFER transmite?	No sobre ARINC. Solo responde por SPI hacia la 3B+ cuando el host le consulta.
SPI o UART/I2C?	SPI da mas margen y menor overhead. I2C no conviene por direccionamiento/latencia; UART seria simple pero menos flexible para paquetes y control.
Por que bridge local?	Para no exponer el control SPI a la red. Flask consume por 127.0.0.1 y la notebook ve solo Flask.
Por que no contar startup como error operativo?	Porque consultar antes de que la Pico responda no mide la calidad del enlace en regimen.
Que representa el rojo del osciloscopio?	La resta CH1 - CH2. Si CH2 esta alto y CH1 bajo, la resta se ve negativa.
Que falta para campo real?	Receptor ARINC, proteccion, alta impedancia, aislamiento/conectores y validacion con fuente ARINC real.

Fuentes internas: PROJECT_CONTEXT.md, README.md, FASE2_ARINC429_LOGIC.md, evidencias de prueba, diagramas internos y capturas de imagenes/.